

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA0620	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	K.Deepika	Reg. No.:	192371042
Section / Batch:	4th year CSE	Date:	5/8/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I _____ K.deepika(192371042) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ K.deepika _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS Online Lab
Ask Gemini
Not secure
simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=586
School
All Bookmarks

SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
KOVVURU DEEPIKA
192371042

QUESTIONS
Q1 Merge Sort 10 marks
0/1 answered

Q1 Merge Sort

10 marks

Analyze the performance of Merge Sort by executing it for different input sizes and recording the execution time for best, average, and worst cases. Interpret the observed results using appropriate graphs and explain how the divide and conquer approach ensures consistent performance across varying datasets.

Your Code

```

1 import time
2 arr = list(map(int, input().split()))
3 key = int(input())
4 low = 0
5 high = len(arr) - 1
6 start = time.perf_counter()
7 found = -1
8 while low <= high:
9     mid = (low + high) // 2
10    if arr[mid] == key:
11        found = mid
12        break
13    elif arr[mid] < key:
14        low = mid + 1
15 end = time.time()
16 if found != -1:
17     print("Element found at position", found + 1)
18 else:
19     print("Element not found")
20 print("Execution time:", end - start)

```

Run
Save
Submit

Input
10 20 30 40 50 60 70 80 90
70

Output
element found at position 7
execution time: 1784671410.9908462

27°C
Mostly cloudy

Search

ENG IN
10:45
05-08-2026